

EF Code First – Modelling Relationships in Code First.

Create a new solution for each exercise.

Use Entity Framework 6 with .NET

You can use a Console app instead of a WPF/XAML app.

Exercise 1 – One to One Relationship

- a) Create a user class with the properties shown below

```
public class User
{
    public int UserId { get; set; }
    public string Username { get; set; }
    public string Email { get; set; }

    // TODO: add a navigation property to UserProfile
}
```

- b) Create the UserProfile class. It should have its own ProfileId, a Bio, an AvatarUrl, and a navigation property back to User.

Hint: Remember: for a 1-to-1 in EF6, either use the shared PK approach (ProfileId = UserId) or keep a separate ProfileId and handle the FK using .Map() in Fluent API.

- c) Create a UserContext class that inherits from DbContext. Add DbSet properties for both User and UserProfile.
- d) Override OnModelCreating and write the Fluent API configuration for the one-to-one relationship. Use .HasOptional() and .WithRequired() to express that a User may optionally have a profile, but a UserProfile must have a User.

```
// Starter – complete the configuration
modelBuilder.Entity<User>()
    .HasOptional( ??? )
    .WithRequired( ??? )
    .Map( ??? );
```

- e) In your Program.cs, create a new User and assign a UserProfile to them. Save both to the database using a single SaveChanges() call.
- f) Add a DateOfBirth property to UserProfile. Create and run a new migration. Verify the column appears in the database table.

Exercise 2 - One-to-Many Relationship

Scenario: A company has many Departments. Each Department can have many Employees, but each Employee belongs to exactly one Department. An Employee must always be assigned to a Department — this is required.

- a) Create the Department class with DepartmentId, Name, and Location properties. Add a collection navigation property for Employees.

Hint: Use ICollection<Employee> and initialise it to new List<Employee>() in the constructor or inline.
- b) Create the Employee class with EmployeeId, FullName, and JobTitle. Add a DepartmentId FK property and a navigation property back to Department.
- c) Create a CompanyContext and configure the one-to-many relationship using Fluent API. An Employee must belong to a Department — make the relationship required.

```
// Starter - complete the configuration
modelBuilder.Entity<Department>()
    .HasMany( ??? )
    .WithRequired( ??? )
    .HasForeignKey( ??? );
```

- d) Run Add-Migration InitialCreate and Update-Database. Open the database and confirm the Employees table has a non-nullable DepartmentId column.
- e) Seed two Departments. Add at least two Employees to each Department. Save to the database.
- f) Write a query that loads all Departments and their Employees. Loop through each Department and print its name, followed by the full name of each Employee in that Department.

Exercise 3 – Many to Many Relationships

Scenario: A college system needs to track Students and Courses. A Student can enrol on many Courses, and a Course can have many Students enrolled on it.

Part A uses EF6's automatic join table. Part B extends this to store an EnrolmentDate

on the enrolment, which requires an explicit join entity.

- a) Create a Student class (StudentId, FullName, StudentNumber) and a Course class (CourseId, Title, Credits). Add a collection navigation property on each class pointing to the other.
- b) Create a CollegeContext and configure the many-to-many using .HasMany().WithMany().Map(). Name the join table StudentCourses and name the key columns StudentId and CourseId.

```
// Starter – complete the configuration
modelBuilder.Entity<Student>()
    .HasMany( ??? )
    .WithMany( ??? )
    .Map(m => {
        m.ToTable( ??? );
        m.MapLeftKey( ??? );
        m.MapRightKey( ??? );
    });
```

- c) Seed three Students and three Courses. Enrol each student on at least two courses. Query and print each Student's name alongside the titles of their enrolled Courses.

Part B — Explicit Join Entity with Extra Column

Why change the approach?

The automatic join table only stores the two foreign keys. If you need to store extra data on the relationship — like an `EnrolmentDate` — you must create an explicit join entity and model it as two one-to-many relationships instead.

- a) Create an `Enrolment` class that acts as the explicit join entity. It should have: `EnrolmentId`, `StudentId`, `CourseId`, `EnrolmentDate`, and navigation properties back to both `Student` and `Course`.
- b) Update `Student` and `Course` to remove the direct collection navigation properties pointing to each other. Replace them with collections of `Enrolment` instead.
- c) In your `DbContext`, configure two one-to-many relationships: `Student` has many `Enrolments`, and `Course` has many `Enrolments`. Both relationships should be required on the `Enrolment` side.

```
// Configure Student -> Enrolments
modelBuilder.Entity<Student>()
    .HasMany( ??? )
    .WithRequired( ??? )
    .HasForeignKey( ??? );

// Configure Course -> Enrolments
modelBuilder.Entity<Course>()
    .HasMany( ??? )
    .WithRequired( ??? )
    .HasForeignKey( ??? );
```

- d) Seed data and write a query that retrieves all students enrolled on the course titled 'Mathematics', printing their name and enrolment date.

Hint: You will need to use `.Include()` on the `Enrolment` to reach both `Student` and `Course`.